

MINI PROJECT REPORT

Spotify Playlist Audio Feature Analysis

Extracting and graphing danceability, energy and tempo across musical genres

Student Name	Your Name
Roll / Enrollment No.	Roll No / Enrollment No
Course	Data Science (PBL)
Institute	Your Institute / Department
Faculty Guide	Faculty Guide Name
Technology	Python, pandas, matplotlib, seaborn, scikit-learn, REST APIs

Abstract

This project builds a reproducible data-science pipeline that collects track-level audio metrics (danceability, energy, valence, acousticness, tempo and loudness) for tracks in Spotify playlists, groups the tracks by genre, and analyses how the metrics differ between genres using descriptive statistics, one-way ANOVA with effect sizes, visualisation, and an unsupervised (PCA + K-Means) check. The pipeline connects to the Spotify Web API for playlist and artist-genre data and to the free ReccoBeats API for audio features, because Spotify closed its own audio-features endpoint to newly created apps on 27 November 2024. An offline mode makes the project runnable without API keys.

Data used for the results in this report: synthetic offline sample dataset (2,000 tracks, 8 genres). The sample data is generated by `src/generate_sample_data.py` from assumed genre profiles; it is **not** real Spotify data, so the numbers below demonstrate that the pipeline works rather than being findings about real music. Running `python main.py --mode api` or `--mode csv` regenerates every table and figure from real data.

Keywords: Spotify Web API, audio features, danceability, energy, tempo, genre analysis, ANOVA, PCA, K-Means.

1. Introduction and Objectives

Streaming platforms describe each track with numeric audio descriptors. Comparing these descriptors across genres helps understand what makes genres sonically distinct and supports tasks such as playlist curation and recommendation. The objectives of this project are:

- Connect to music APIs and extract playlist tracks, artist genres and audio features programmatically.
- Clean and combine the data into one analysis-ready table.
- Graph danceability, energy and tempo (plus supporting metrics) across genres.
- Test statistically whether the differences between genres are significant and how large they are.
- Check whether audio features alone can recover genre groupings (clustering).

2. Audio Features Used

Feature	Range	Meaning
danceability	0 - 1	Suitability for dancing (tempo stability, beat strength, regularity)
energy	0 - 1	Perceived intensity and activity (fast, loud, noisy tracks score high)
valence	0 - 1	Musical positiveness (high = happy/cheerful, low = sad/tense)
acousticness	0 - 1	Confidence the track is acoustic
tempo	BPM	Estimated beats per minute
loudness	dB	Overall loudness (typically -60 to 0 dB)

3. System Design and Methodology

The project is organised as follows:

```
spotify_audio_analysis/  
  main.py           CLI entry point (--mode sample | csv | api)  
  config.py         paths, feature lists, genres  
  requirements.txt  .env.example
```

src/data_sources.py	Spotify + ReccoBeats API clients, CSV loader
src/generate_sample_data.py	offline synthetic dataset
src/analysis.py	cleaning, statistics, 7 graphs, clustering
report/build_report.py	builds this PDF
data/ outputs/	inputs and results (tables, figures)

Pipeline. (1) Authenticate to Spotify with the Client Credentials flow. (2) Read every track of each public playlist, following pagination. (3) Look up the primary genre of each track's first artist (Spotify stores genres at artist level, not track level). (4) Request audio features: Spotify's endpoint is tried first; on HTTP 403 the code falls back to ReccoBeats. (5) Merge, drop missing values, duplicates and failed tempo detections (tempo = 0), and keep genres with at least 15 tracks. (6) Compute per-genre statistics, ANOVA and eta-squared, draw graphs, and run PCA + K-Means.

Statistics. For each feature a one-way ANOVA tests whether genre means differ. Because p-values become tiny with large samples, eta-squared (share of variance explained by genre) is reported as the effect size. K-Means (k = number of genres) is run on standardised features and scored by cluster purity against the true genre.

API limitations (important). Spotify's audio-features, audio-analysis and recommendations endpoints return HTTP 403 for apps created after 27 November 2024, and Spotify-owned editorial playlists are also restricted for new apps. The code therefore uses user-created public playlists and a documented fallback; ReccoBeats coverage is not guaranteed, so tracks it does not return are dropped and the final track count is reported.

4. Results and Outcome

After cleaning, the analysis used **2,000** tracks across **8** genres (metal, electronic, rock, pop, hip-hop, country, jazz, classical).

4.1 Genre summary (means)

Genre	N	Dance.	Energy	Tempo	Valence	Acoust.	Loud. (dB)
metal	250	0.38	0.90	134	0.35	0.03	-5.54
electronic	250	0.70	0.77	125	0.44	0.10	-5.79
rock	250	0.51	0.74	127	0.52	0.13	-6.69
pop	250	0.64	0.65	117	0.53	0.22	-6.44
hip-hop	250	0.73	0.63	122	0.50	0.17	-6.81
country	250	0.57	0.60	120	0.59	0.30	-7.64
jazz	250	0.49	0.34	111	0.51	0.67	-14.17
classical	250	0.30	0.20	107	0.25	0.91	-22.39

4.2 Danceability, energy and tempo across genres

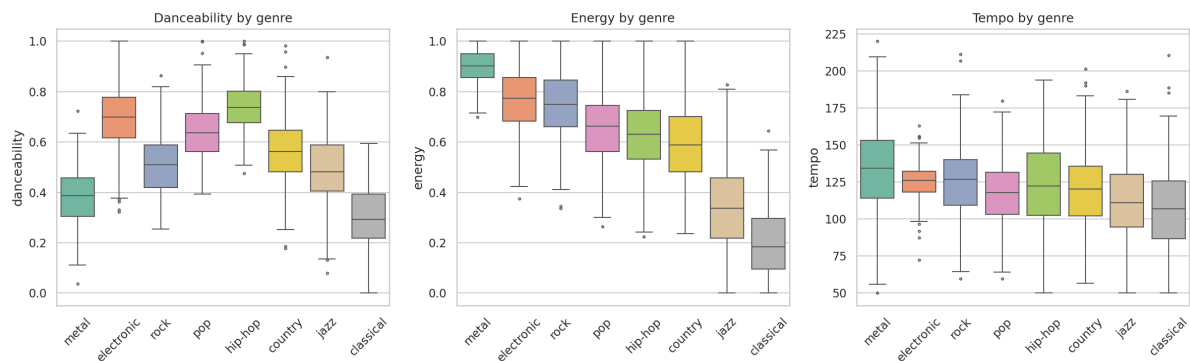


Figure 1. Distribution of the three headline metrics by genre (sorted by median energy).

Energy is highest for **metal** and lowest for **classical**. **Danceability** is highest for **hip-hop** and lowest for **classical**. Mean **tempo** is highest for **metal** and lowest for **classical**, but tempo distributions overlap heavily.

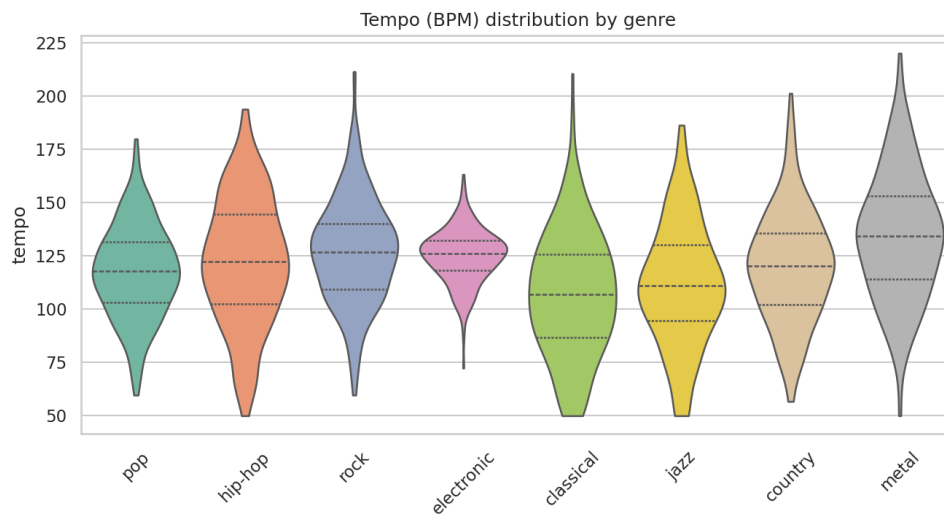


Figure 2. Tempo (BPM) distribution by genre (violin plot with quartiles).

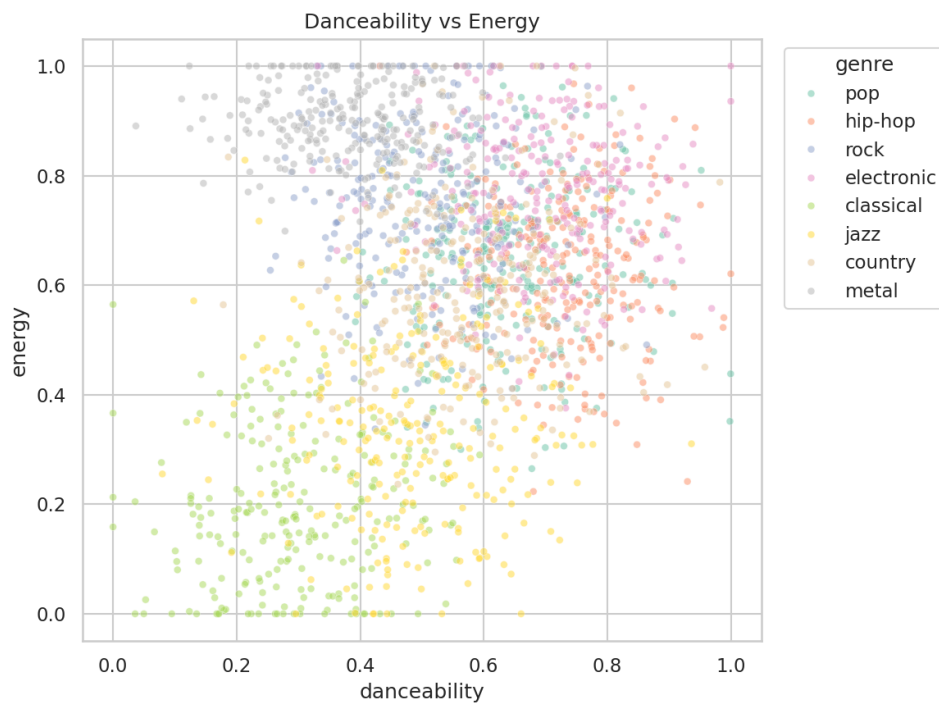


Figure 3. Danceability versus energy; genres occupy different regions of the plane.

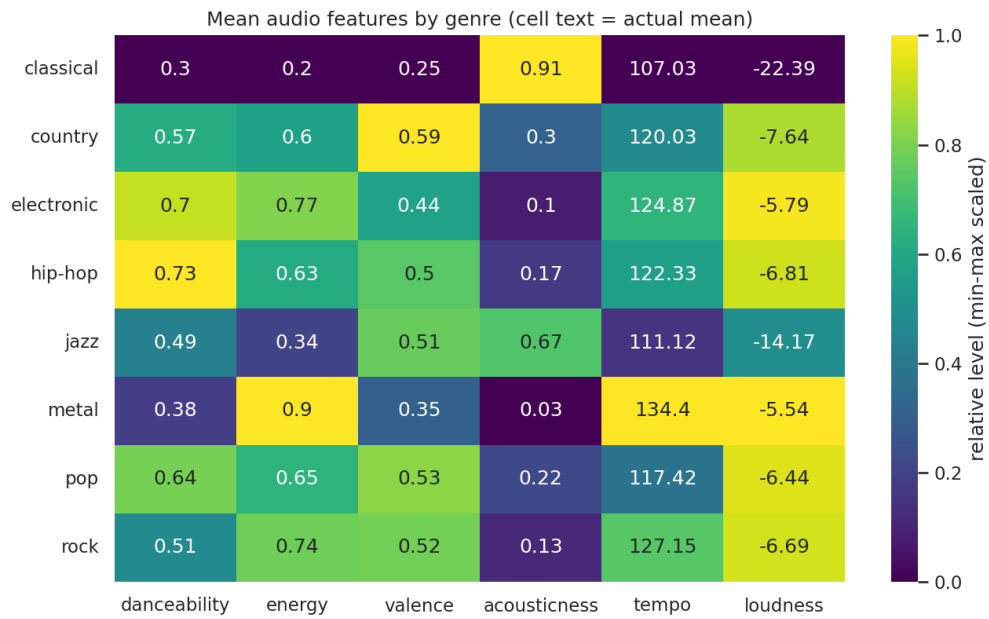


Figure 4. Mean value of every feature by genre (colour = min-max scaled, text = real mean).

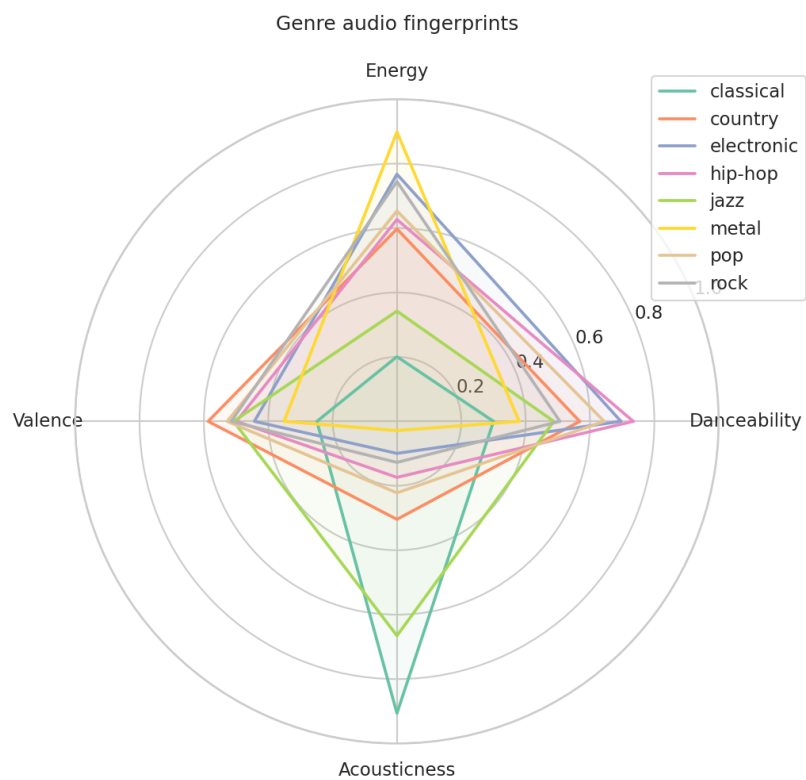


Figure 5. Radar chart of each genre's four 0-1 features.

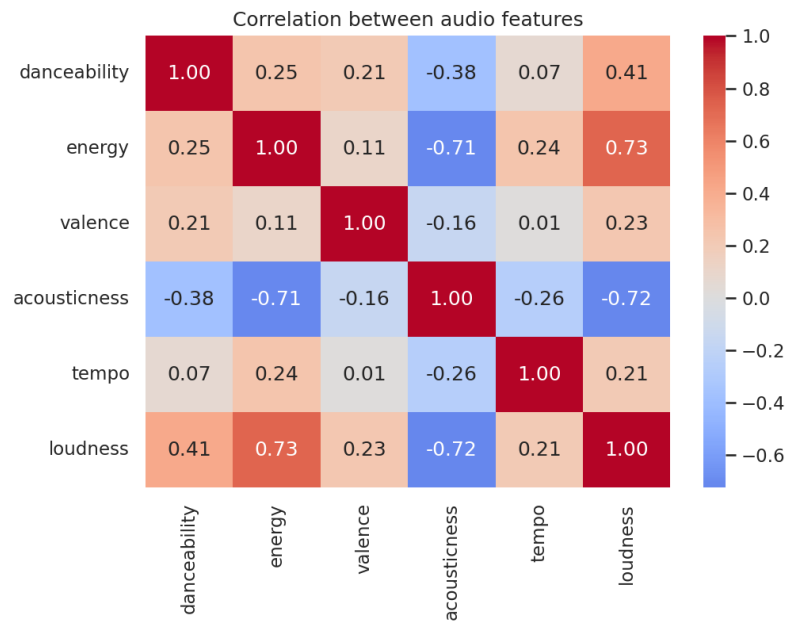


Figure 6. Pearson correlations between features.

4.3 Statistical tests

Feature	F	p-value	eta-squared
danceability	406.8	< 1e-30	0.588
energy	661.8	< 1e-30	0.699
valence	73.7	< 1e-30	0.206
acousticness	1050.1	< 1e-30	0.787
tempo	28.8	< 1e-30	0.092
loudness	746.8	< 1e-30	0.724

All differences between genres are statistically significant ($p < 0.001$). The effect sizes show that genre explains most of the variation in acousticness, loudness and energy, a large share of danceability, and comparatively little of tempo. In other words, tempo is the weakest single descriptor for separating genres.

4.4 Correlations and clustering

Energy correlates positively with loudness ($r = 0.728$) and negatively with acousticness ($r = -0.711$). Danceability and valence are only weakly related ($r = 0.211$). K-Means on the standardised features reaches a cluster purity of 0.483 against the true genres (chance level for 8 equal genres is about 0.125), so audio features carry real genre information, but genres with similar profiles (e.g. pop, hip-hop, country, rock) remain hard to separate without extra features.

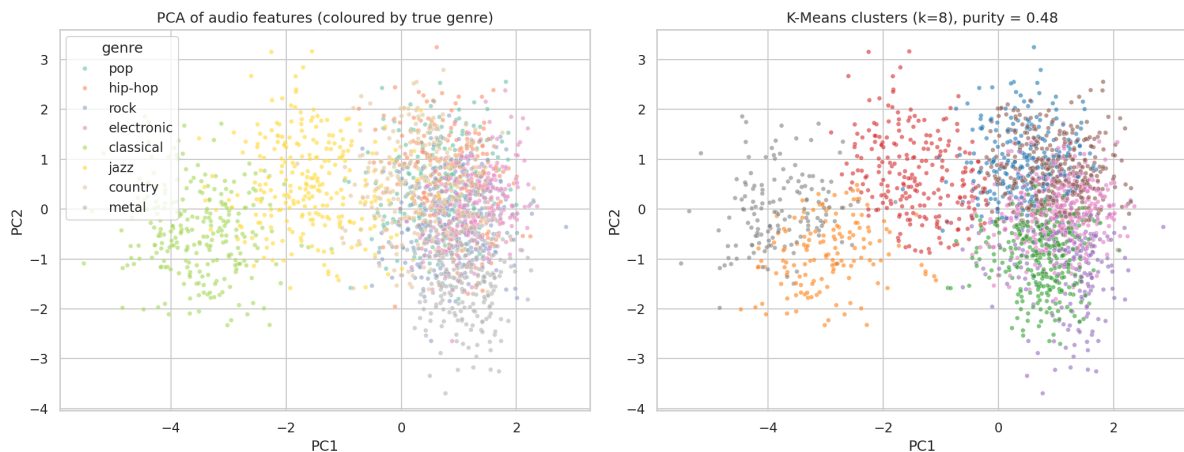


Figure 7. PCA projection coloured by genre (left) and by K-Means cluster (right).

5. Conclusion

The project delivers a working, reproducible pipeline from API extraction to statistical analysis and graphs. On the data used here, energy, acousticness and loudness separate genres most strongly, danceability separates them moderately, and tempo least. Because this report's numbers come from a synthetic sample, the conclusions describe the expected behaviour of the method; the same code should be run on real playlists to draw conclusions about real music.

Limitations. Spotify's audio features are model estimates, not ground truth; genre is assigned per artist (an artist's first listed genre), so individual tracks can be mislabelled; ReccoBeats coverage and consistency with Spotify's original values are not guaranteed; playlists are a biased sample of music.

Future work. Add a supervised genre classifier, compare decades or markets, add time-series analysis of a listener's playlist, and build a Streamlit dashboard.

References

- Spotify for Developers, Web API reference: developer.spotify.com/documentation/web-api
- Spotify for Developers blog, "Changes to Web API" (27 Nov 2024): developer.spotify.com/blog/2024-11-27-changes-to-the-web-api
- ReccoBeats API documentation: reccobeats.com/docs
- pandas, matplotlib, seaborn, SciPy and scikit-learn documentation.

Appendix A: Source Code

main.py

```
"""
Spotify Playlist Audio Feature Analysis -- command line entry point.

Examples
python main.py --mode sample
python main.py --mode csv --csv data/dataset.csv
python main.py --mode api --playlists https://open.spotify.com/playlist/XXXX https://open.spotify.com/playlist/YY
YY
"""
import argparse

from src.analysis import run_all
from src.data_sources import load_data

def main():
    p = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    p.add_argument("--mode", choices=["sample", "csv", "api"], default="sample")
    p.add_argument("--csv", help="path to CSV (mode=csv)")
    p.add_argument("--playlists", nargs="+", help="Spotify playlist URLs/IDs (mode=api)")
    a = p.parse_args()

    df = load_data(a.mode, a.playlists, a.csv)
    print(f"Loaded {len(df)} rows from mode='{a.mode}'")
    res = run_all(df, label=a.mode)

    print("\n=== KEY RESULTS ===")
    for k, v in res.items():
        print(f"{k:28s}: {v}")
    print("\nFigures -> outputs/figures/   Tables -> outputs/*.csv   Summary -> outputs/results.json")

if __name__ == "__main__":
    main()
```

config.py

```
"""Central configuration for the Spotify Audio Feature Analysis project."""
from pathlib import Path

ROOT = Path(__file__).resolve().parent
DATA_DIR = ROOT / "data"
OUT_DIR = ROOT / "outputs"
FIG_DIR = OUT_DIR / "figures"

SAMPLE_CSV = DATA_DIR / "sample_tracks.csv"
API_CSV = DATA_DIR / "api_tracks.csv" # written by --mode api

# Audio metrics analysed in the project
FEATURES = ["danceability", "energy", "valence", "acousticness", "tempo", "loudness"]
CORE_FEATURES = ["danceability", "energy", "tempo"] # the three headline metrics

# Genres analysed in the offline sample (also a good set for real playlists)
GENRES = ["pop", "hip-hop", "rock", "electronic", "classical", "jazz", "country", "metal"]

RANDOM_STATE = 42
```

src/data_sources.py

```
"""
Data loading for the project. Three modes:

sample : synthetic offline dataset (no keys, no internet)
csv     : any CSV with genre + audio-feature columns (e.g. the Kaggle
          "Spotify Tracks Dataset" which has a `track_genre` column)
api     : live pull -- playlist tracks + artist genres from the Spotify Web API,
          audio features from Spotify (if your app still has access) or from the
          free ReccoBeats API (fallback).

NOTE: Spotify closed /v1/audio-features to apps created after 27 Nov 2024 (HTTP 403),
which is why the ReccoBeats fallback exists.
"""
import base64
import os
import re
import time

import pandas as pd
import requests

from config import API_CSV, FEATURES, SAMPLE_CSV

SPOTIFY_TOKEN_URL = "https://accounts.spotify.com/api/token"
SPOTIFY_API = "https://api.spotify.com/v1"
RECCO_API = "https://api.reccobeats.com/v1"
```

```

FEATURE_COLS = ["danceability", "energy", "valence", "acousticness", "tempo", "loudness"]
# ----- helpers
def _get(url, headers=None, params=None, retries=4):
    """GET with simple retry/back-off (handles HTTP 429 rate limits)."""
    for attempt in range(retries):
        r = requests.get(url, headers=headers, params=params, timeout=30)
        if r.status_code == 429:
            wait = int(r.headers.get("Retry-After", 2 ** attempt))
            time.sleep(wait)
            continue
        return r
    return r

def _chunks(seq, n):
    for i in range(0, len(seq), n):
        yield seq[i:i + n]

def parse_playlist_id(url_or_id: str) -> str:
    m = re.search(r"playlist/([A-Za-z0-9]+)", url_or_id)
    return m.group(1) if m else url_or_id.strip()

# ----- spotify
def get_spotify_token() -> str:
    from dotenv import load_dotenv
    load_dotenv()
    cid, secret = os.getenv("SPOTIFY_CLIENT_ID"), os.getenv("SPOTIFY_CLIENT_SECRET")
    if not cid or not secret:
        raise RuntimeError("Set SPOTIFY_CLIENT_ID and SPOTIFY_CLIENT_SECRET in a .env file "
                           "(see .env.example).")
    auth = base64.b64encode(f"{cid}:{secret}".encode()).decode()
    r = requests.post(SPOTIFY_TOKEN_URL, data={"grant_type": "client_credentials"},
                     headers={"Authorization": f"Basic {auth}"}, timeout=30)
    r.raise_for_status()
    return r.json()["access_token"]

def fetch_playlist_tracks(playlist_id: str, token: str) -> pd.DataFrame:
    """Return track_id, track_name, artist, artist_id, popularity for a PUBLIC playlist."""
    headers = {"Authorization": f"Bearer {token}"}
    url, params, rows = f"{SPOTIFY_API}/playlists/{playlist_id}/tracks", {"limit": 100}, []
    while url:
        r = _get(url, headers, params)
        if r.status_code == 403:
            raise RuntimeError("403 from Spotify: Spotify-owned/editorial playlists are blocked for "
                               "new apps. Use a public playlist created by a normal user.")
        r.raise_for_status()
        data = r.json()
        for item in data["items"]:
            t = item.get("track")
            if not t or not t.get("id"):
                continue
            rows.append(dict(track_id=t["id"], track_name=t["name"],
                             artist=t["artists"][0]["name"], artist_id=t["artists"][0]["id"],
                             popularity=t.get("popularity")))
        url, params = data.get("next"), None
    return pd.DataFrame(rows).drop_duplicates("track_id")

def fetch_artist_genres(artist_ids, token: str) -> dict:
    """Map artist_id -> primary genre string (first genre Spotify lists, else 'unknown')."""
    headers, out = {"Authorization": f"Bearer {token}"}, {}
    for batch in _chunks(list(dict.fromkeys(artist_ids)), 50):
        r = _get(f"{SPOTIFY_API}/artists", headers, {"ids": ",".join(batch)})
        if r.status_code != 200:
            continue
        for a in r.json().get("artists", []):
            if a:
                out[a["id"]] = a["genres"][0] if a.get("genres") else "unknown"
    return out

def fetch_audio_features_spotify(track_ids, token: str):
    """Try Spotify's own endpoint. Returns DataFrame, or None if blocked (403)."""
    headers, rows = {"Authorization": f"Bearer {token}"}, []
    for batch in _chunks(list(track_ids), 100):
        r = _get(f"{SPOTIFY_API}/audio-features", headers, {"ids": ",".join(batch)})
        if r.status_code in (403, 404):
            return None
        r.raise_for_status()
        rows += [f for f in r.json().get("audio_features", []) if f]
    if not rows:
        return None
    df = pd.DataFrame(rows).rename(columns={"id": "track_id"})
    return df[["track_id"] + FEATURE_COLS]

def fetch_audio_features_reccobeats(track_ids) -> pd.DataFrame:
    """
    Free fallback: ReccoBeats returns Spotify-style audio features.
    Docs: https://reccobeats.com/docs (endpoint accepts Spotify track IDs; each
    result carries an `href` containing the Spotify URL, which we use to map back).
    Coverage is not 100%, so missing tracks are simply dropped.
    """
    rows = []
    for batch in _chunks(list(track_ids), 40):
        r = _get(f"{RECCO_API}/audio-features", params={"ids": ",".join(batch)})
        if r.status_code != 200:
            print(f"ReccoBeats returned HTTP {r.status_code} for a batch; skipping it.")

```

```

        continue
    for item in r.json().get("content", []):
        m = re.search(r"track/([A-Za-z0-9]+)", item.get("href", ""))
        if m:
            rec = {k: item.get(k) for k in FEATURE_COLS}
            rec["track_id"] = m.group(1)
            rows.append(rec)
        time.sleep(0.3) # be polite to a free API
    return pd.DataFrame(rows)

def load_from_api(playlist_urls) -> pd.DataFrame:
    token = get_spotify_token()
    frames = []
    for url in playlist_urls:
        pid = parse_playlist_id(url)
        print(f"Fetching playlist {pid} ...")
        frames.append(fetch_playlist_tracks(pid, token))
    tracks = pd.concat(frames).drop_duplicates("track_id").reset_index(drop=True)
    print(f" {len(tracks)} unique tracks")

    genres = fetch_artist_genres(tracks["artist_id"], token)
    tracks["genre"] = tracks["artist_id"].map(genres).fillna("unknown")

    feats = fetch_audio_features_spotify(tracks["track_id"], token)
    if feats is None:
        print("Spotify audio-features is blocked for this app (403) -> using ReccoBeats fallback")
        feats = fetch_audio_features_reccobeats(tracks["track_id"])
    if feats.empty:
        raise RuntimeError("No audio features could be retrieved. Try --mode csv instead.")

    df = tracks.merge(feats, on="track_id", how="inner")
    print(f" {len(df)} tracks with audio features")
    df.to_csv(API_CSV, index=False)
    print(f" saved -> {API_CSV}")
    return df

# ----- CSV
def load_csv(path) -> pd.DataFrame:
    df = pd.read_csv(path)
    if "genre" not in df.columns and "track_genre" in df.columns: # Kaggle naming
        df = df.rename(columns={"track_genre": "genre"})
    if "track_name" not in df.columns and "name" in df.columns:
        df = df.rename(columns={"name": "track_name"})
    missing = [c for c in ["genre"] + FEATURE_COLS if c not in df.columns]
    if missing:
        raise ValueError(f"CSV is missing required columns: {missing}")
    return df

def load_data(mode: str, playlists=None, csv_path=None) -> pd.DataFrame:
    if mode == "sample":
        if not SAMPLE_CSV.exists():
            from src.generate_sample_data import generate
            SAMPLE_CSV.parent.mkdir(exist_ok=True)
            generate().to_csv(SAMPLE_CSV, index=False)
        return load_csv(SAMPLE_CSV)
    if mode == "csv":
        if not csv_path:
            raise ValueError("--csv PATH is required with --mode csv")
        return load_csv(csv_path)
    if mode == "api":
        if not playlists:
            raise ValueError("--playlists URL [URL ...] is required with --mode api")
        return load_from_api(playlists)
    raise ValueError(f"Unknown mode: {mode}")

```

src/generate_sample_data.py

```

"""
Generate a SYNTHETIC offline dataset so the project runs without API keys.

The per-genre means / spreads below are loosely based on well-known tendencies
of Spotify's audio features (e.g. classical = low energy & high acousticness,
metal = very high energy & loudness, hip-hop = high speechiness/danceability).
They are NOT real measurements -- for real results use --mode api or --mode csv.
"""
import numpy as np
import pandas as pd

from config import GENRES, RANDOM_STATE, SAMPLE_CSV

# genre: {feature: (mean, std)}
PROFILES = {
    "pop": dict(danceability=(0.65, 0.12), energy=(0.65, 0.15), valence=(0.55, 0.20), acousticness=(0.20, 0.20), tempo=(118, 22), loudness=(-6.5, 2.0)),
    "hip-hop": dict(danceability=(0.75, 0.10), energy=(0.65, 0.14), valence=(0.50, 0.20), acousticness=(0.15, 0.17), tempo=(120, 30), loudness=(-6.8, 2.2)),
    "rock": dict(danceability=(0.50, 0.12), energy=(0.75, 0.15), valence=(0.50, 0.22), acousticness=(0.12, 0.17), tempo=(125, 25), loudness=(-6.5, 2.5)),
    "electronic": dict(danceability=(0.70, 0.12), energy=(0.78, 0.13), valence=(0.45, 0.22), acousticness=(0.07, 0.12), tempo=(125, 12), loudness=(-6.0, 2.0)),
    "classical": dict(danceability=(0.30, 0.12), energy=(0.18, 0.15), valence=(0.25, 0.18), acousticness=(0.92,

```

```

        0.10), tempo=(105, 32), loudness=(-22, 6.0)),
    "jazz": dict(danceability=(0.50, 0.14), energy=(0.35, 0.18), valence=(0.50, 0.22), acousticness=(0.65,
    0.25), tempo=(112, 30), loudness=(-14, 4.5)),
    "country": dict(danceability=(0.57, 0.12), energy=(0.62, 0.16), valence=(0.58, 0.22), acousticness=(0.30,
    0.25), tempo=(122, 26), loudness=(-7.5, 2.5)),
    "metal": dict(danceability=(0.38, 0.11), energy=(0.90, 0.07), valence=(0.35, 0.20), acousticness=(0.02,
    0.05), tempo=(133, 30), loudness=(-5.5, 2.0)),
}
BOUNDS = dict(danceability=(0, 1), energy=(0, 1), valence=(0, 1), acousticness=(0, 1),
    tempo=(50, 220), loudness=(-40, 0))

def generate(n_per_genre: int = 250, seed: int = RANDOM_STATE) -> pd.DataFrame:
    rng = np.random.default_rng(seed)
    rows = []
    for genre in GENRES:
        data = {"genre": genre}
        cols = {}
        for feat, (mu, sd) in PROFILES[genre].items():
            lo, hi = BOUNDS[feat]
            cols[feat] = np.clip(rng.normal(mu, sd, n_per_genre), lo, hi)
        # realistic coupling: louder tracks tend to be more energetic
        cols["loudness"] = np.clip(cols["loudness"] + 6 * (cols["energy"] - PROFILES[genre]["energy"][0]), -40, 0)
        df = pd.DataFrame(cols)
        df["genre"] = genre
        rows.append(df)
    df = pd.concat(rows, ignore_index=True)
    df.insert(0, "track_id", [f"SAMPLE{i:05d}" for i in range(len(df))])
    df.insert(1, "track_name", [f"{g.title()} Track {i}" for i, g in enumerate(df["genre"])]])
    df["artist"] = "Synthetic Artist"
    df["popularity"] = np.clip(rng.normal(50, 18, len(df)), 0, 100).round().astype(int)
    return df.round(4)

if __name__ == "__main__":
    df = generate()
    SAMPLE_CSV.parent.mkdir(exist_ok=True)
    df.to_csv(SAMPLE_CSV, index=False)
    print(f"Wrote {len(df)} synthetic tracks -> {SAMPLE_CSV}")

```

src/analysis.py

```

"""Cleaning, statistics and graphing of audio features across genres."""
import json

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from scipy import stats
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

from config import CORE_FEATURES, FEATURES, FIG_DIR, OUT_DIR, RANDOM_STATE

sns.set_theme(style="whitegrid", context="notebook")
PALETTE = "Set2"

# ----- cleaning
def clean(df: pd.DataFrame, top_n_genres: int = 8, min_tracks: int = 15) -> pd.DataFrame:
    df = df.copy()
    df = df.dropna(subset=FEATURES + ["genre"]).drop_duplicates(subset=[c for c in ["track_id"] if c in df])
    df = df[(df["tempo"] > 0)] # tempo 0 = failed detection
    df = df[df["genre"].str.lower() != "unknown"]
    counts = df["genre"].value_counts()
    keep = counts[counts >= min_tracks].head(top_n_genres).index
    return df[df["genre"].isin(keep)].reset_index(drop=True)

# ----- statistics
def summary_table(df):
    s = df.groupby("genre")[FEATURES].agg(["mean", "std"]).round(3)
    s.columns = [f"{a}_{b}" for a, b in s.columns]
    s.insert(0, "n_tracks", df.groupby("genre").size())
    return s.sort_values("energy_mean", ascending=False)

def anova_tests(df):
    """One-way ANOVA + eta-squared effect size for each feature across genres."""
    rows = []
    for f in FEATURES:
        groups = [g[f].values for _, g in df.groupby("genre")]
        F, p = stats.f_oneway(*groups)
        grand = df[f].mean()
        ss_between = sum(len(g) * (g.mean() - grand) ** 2 for g in groups)
        ss_total = ((df[f] - grand) ** 2).sum()
        rows.append(dict(feature=f, F=round(F, 1), p_value=float(p), eta_squared=round(ss_between / ss_total, 3)))
    return pd.DataFrame(rows)

# ----- plots
def _save(fig, name):
    FIG_DIR.mkdir(parents=True, exist_ok=True)
    path = FIG_DIR / name
    fig.tight_layout()

```

```

fig.savefig(path, dpi=150)
plt.close(fig)
return path

def plot_boxplots(df):
    order = df.groupby("genre")["energy"].median().sort_values(ascending=False).index
    fig, axes = plt.subplots(1, 3, figsize=(16, 5))
    for ax, f in zip(axes, CORE_FEATURES):
        sns.boxplot(data=df, x="genre", y=f, order=order, hue="genre", hue_order=order, palette=PALETTE,
                    legend=False, ax=ax, fliersize=2)
        ax.set_title(f"{f.title()} by genre")
        ax.tick_params(axis="x", rotation=45)
        ax.set_xlabel("")
    return _save(fig, "01_boxplots_core_features.png")

def plot_heatmap(df):
    means = df.groupby("genre")[FEATURES].mean()
    scaled = (means - means.min()) / (means.max() - means.min()) # min-max per feature for comparability
    fig, ax = plt.subplots(figsize=(9, 5.5))
    sns.heatmap(scaled, annot=means.round(2), fmt="", cmap="viridis",
                cbar_kws={"label": "relative level (min-max scaled)", ax=ax})
    ax.set_title("Mean audio features by genre (cell text = actual mean)")
    ax.set_ylabel("")
    return _save(fig, "02_genre_feature_heatmap.png")

def plot_scatter(df):
    fig, ax = plt.subplots(figsize=(8, 6))
    sns.scatterplot(data=df, x="danceability", y="energy", hue="genre", palette=PALETTE, alpha=0.5, s=18, ax=ax)
    ax.set_title("Danceability vs Energy")
    ax.legend(bbox_to_anchor=(1.02, 1), loc="upper left", title="genre")
    return _save(fig, "03_danceability_vs_energy.png")

def plot_tempo_distribution(df):
    fig, ax = plt.subplots(figsize=(9, 5))
    sns.violinplot(data=df, x="genre", y="tempo", hue="genre", palette=PALETTE, legend=False, inner="quartile",
                  cut=0, ax=ax)
    ax.set_title("Tempo (BPM) distribution by genre")
    ax.tick_params(axis="x", rotation=45)
    ax.set_xlabel("")
    return _save(fig, "04_tempo_distribution.png")

def plot_correlation(df):
    fig, ax = plt.subplots(figsize=(7, 5.5))
    sns.heatmap(df[FEATURES].corr(), annot=True, fmt=".2f", cmap="coolwarm", center=0, ax=ax)
    ax.set_title("Correlation between audio features")
    return _save(fig, "05_correlation_matrix.png")

def plot_radar(df):
    cols = ["danceability", "energy", "valence", "acousticness"]
    means = df.groupby("genre")[cols].mean()
    angles = np.linspace(0, 2 * np.pi, len(cols), endpoint=False).tolist()
    angles += angles[:1]
    fig, ax = plt.subplots(figsize=(7, 7), subplot_kw=dict(polar=True))
    colors = sns.color_palette(PALETTE, len(means))
    for color, (genre, row) in zip(colors, means.iterrows()):
        vals = row.tolist() + row.tolist()[:1]
        ax.plot(angles, vals, label=genre, color=color, linewidth=1.8)
        ax.fill(angles, vals, color=color, alpha=0.06)
    ax.set_xticks(angles[:-1])
    ax.set_xticklabels([c.title() for c in cols])
    ax.set_ylim(0, 1)
    ax.set_title("Genre audio fingerprints", pad=20)
    ax.legend(bbox_to_anchor=(1.15, 1.05))
    return _save(fig, "06_radar_genre_profiles.png")

def plot_clusters(df):
    """Unsupervised check: do tracks cluster by genre using audio features alone?"""
    X = StandardScaler().fit_transform(df[FEATURES])
    k = df["genre"].nunique()
    labels = KMeans(n_clusters=k, n_init=10, random_state=RANDOM_STATE).fit_predict(X)
    coords = PCA(n_components=2, random_state=RANDOM_STATE).fit_transform(X)
    purity = (pd.crosstab(labels, df["genre"]).max(axis=1).sum()) / len(df)
    fig, axes = plt.subplots(1, 2, figsize=(14, 5.5))
    sns.scatterplot(x=coords[:, 0], y=coords[:, 1], hue=df["genre"], palette=PALETTE, s=14, alpha=0.6, ax=axes[0])
    axes[0].set_title("PCA of audio features (coloured by true genre)")
    sns.scatterplot(x=coords[:, 0], y=coords[:, 1], hue=labels, palette="tab10", s=14, alpha=0.6, ax=axes[1],
                  legend=False)
    axes[1].set_title(f"K-Means clusters (k={k}), purity = {purity:.2f}")
    for ax in axes:
        ax.set_xlabel("PC1"); ax.set_ylabel("PC2")
    _save(fig, "07_pca_kmeans_clusters.png")
    return round(float(purity), 3)

# ----- pipeline
def run_all(df: pd.DataFrame, label: str = "") -> dict:
    OUT_DIR.mkdir(exist_ok=True)
    df = clean(df)
    summ = summary_table(df)
    anova = anova_tests(df)
    summ.to_csv(OUT_DIR / "genre_summary_stats.csv")
    anova.to_csv(OUT_DIR / "anova_results.csv", index=False)
    df.to_csv(OUT_DIR / "cleaned_dataset.csv", index=False)

    plot_boxplots(df); plot_heatmap(df); plot_scatter(df); plot_tempo_distribution(df)
    plot_correlation(df); plot_radar(df)

```

```

purity = plot_clusters(df)

corr = df[FEATURES].corr()
results = dict(
    source=label, n_tracks=int(len(df)), n_genres=int(df["genre"].nunique()),
    genres=list(summ.index),
    corr_energy_loudness=round(float(corr.loc["energy", "loudness"]), 3),
    corr_energy_acousticness=round(float(corr.loc["energy", "acousticness"]), 3),
    corr_danceability_valence=round(float(corr.loc["danceability", "valence"]), 3),
    kmeans_purity=purity,
    most_danceable=summ["danceability_mean"].idxmax(), least_danceable=summ["danceability_mean"].idxmin(),
    highest_energy=summ["energy_mean"].idxmax(), lowest_energy=summ["energy_mean"].idxmin(),
    fastest=summ["tempo_mean"].idxmax(), slowest=summ["tempo_mean"].idxmin(),
)
(OUT_DIR / "results.json").write_text(json.dumps(results, indent=2))
return results

```